



CS240 Algorithm Design and Analysis

Lecture 0

Introduction and Overview

Qingying Hao

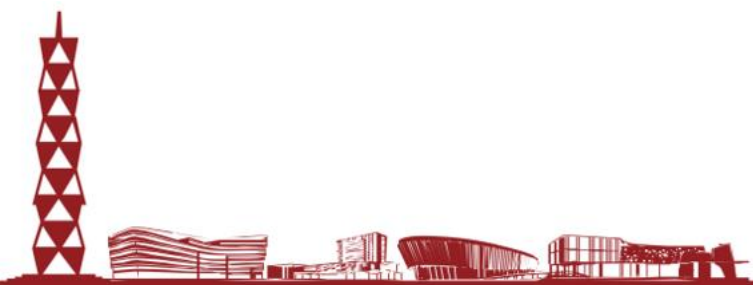
Spring 2026
2026.03.03

Instructor:

- Qingying Hao 郝卿颖
- Office: Room 503E, 1A
- Email: haoqy@shanghaitech.edu.cn

TAs:

- 祖伟钦 : zuwq2025@shanghaitech.edu.cn
- 周珊珊 : zhoushsh2024@shanghaitech.edu.cn
- 何书山 : heshsh2024@shanghaitech.edu.cn
- 赵柏宇 : zhaoby2023@shanghaitech.edu.cn



Prerequisites

- **Algorithm and Data Structure (Undergraduate course)**
 - Sorting and searching, divide & conquer, greedy, dynamic programming, graph algorithms
 - Analysis of algorithms
- **Basic discrete mathematics**
 - Recurrences, logic and proofs, basic graph theory
- **Basic probability theory**
 - Probability space, random variables, expectation, variance
- **Computer programming**
 - Doesn't matter which language(s) you know
 - But you should be capable of translating high-level algorithm descriptions into working programs in some programming language



Organization

- **Textbook**

- [KT] Algorithm Design, by Jon Kleinberg and Eva Tardos.
- [CLRS] Introduction to Algorithms (3rd edition), by T. Cormen, C. Leiserson, R. Rivest, and C. Stein.
- [V] Approximation Algorithms, by Vijay V. Vazirani.
- [MR] Randomized Algorithms, by Rajeev Motwani and Prabhakar Raghavan.

- **Piazza:** <https://piazza.com/shanghaitech.edu.cn/spring2026/cs240> **(Please JOIN as students!)**

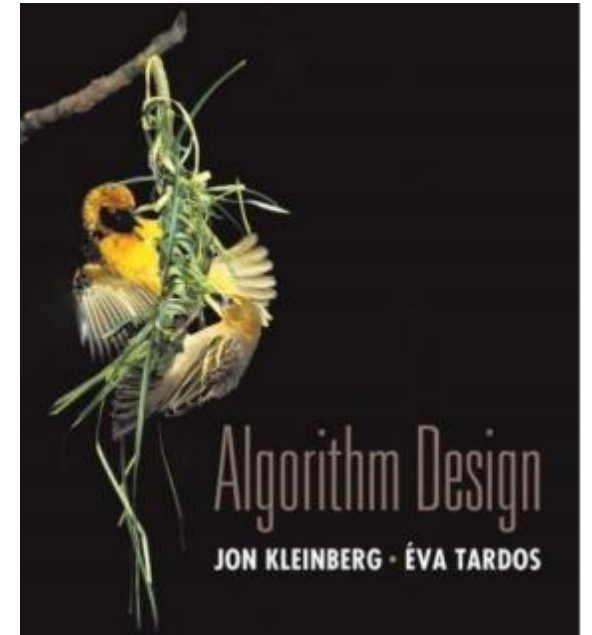
- Lecture slides, announcements, homework assignment, QA and discussions, etc.

- **Gradescope**

- Homework submission and grading
- GradeScope Entry Code: **D6GK3X**

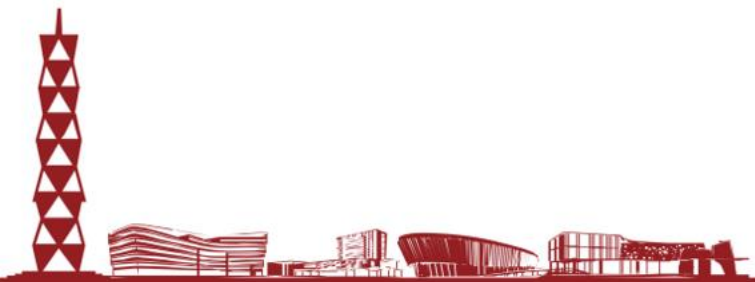
- **Academic Integrity**

- Unless explicitly noted, work turned in should reflect your own/independent capabilities
- No cheating (We will check carefully!)
 - Don't share your homework/code!
 - No fake solutions!
 - No plagiarism!
 - Serious consequences!



Organization

- Grading
 - Assignments (20%)
 - Midterm (35%)
 - Final (35%)
 - Course Project (10%)
 - Exams will be open-book with only 1 A4 cheat sheet double-sided

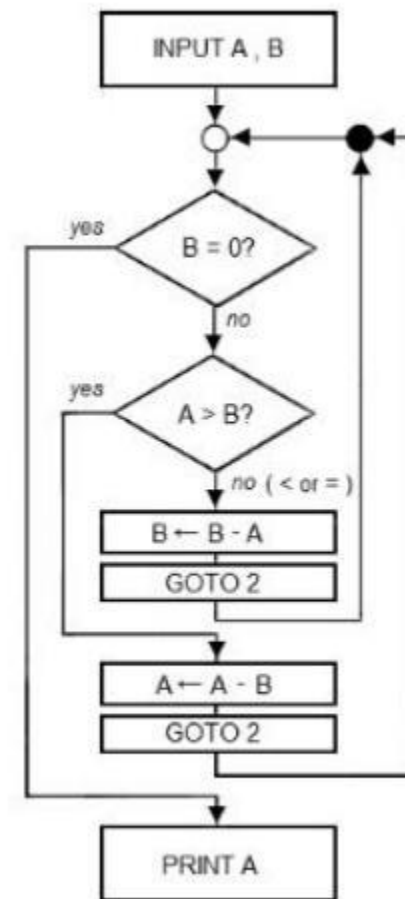




Algorithms



- [Knuth, TAOCP] An algorithm is a finite, definite, effective procedure, with some input and some output
- [Wikipedia] An algorithm is a finite sequence of well-defined, computer-implementable instructions, typically to solve a class of problems or to perform a computation
- Important for all other branches of computer science
- Provides novel “lens” on processes outside of computer science and technology
 - Internet: Web search, packet routing, distributed file sharing, ...
 - Biology: Human genome project, protein folding, ...
 - Computers: Circuit layout, databases, caching, networking, compilers, ...
 - Computer graphics: Movies, video games, virtual reality, ...
 - Security: Cell phones, e-commerce, voting machines, federated learning, ...
 - Multimedia: MP3, JPG, DivX, HDTV, face recognition, ...
 - Social networks: Recommendations, news feeds, advertisements, ...
 - Physics. N-body simulation, particle collision simulation, ...



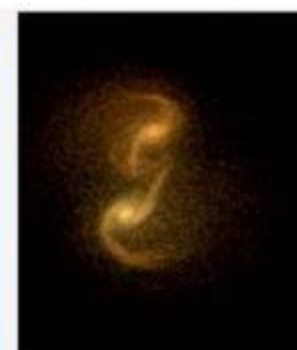
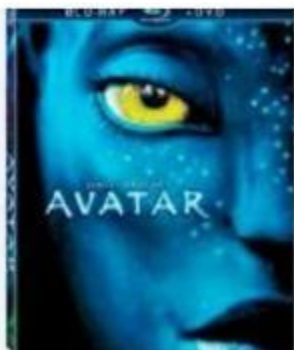


Why Study Algorithms?



上海科技大学
ShanghaiTech University

- Wide range of applications
 - **Internet.** Web search, packet routing, distributed file sharing, ...
 - **Biology.** Human genome project, protein folding, ...
 - **Computers.** Circuit layout, databases, caching, networking, compilers, ...
 - **Computer graphics.** Movies, video games, virtual reality, ...
 - **Security.** Cell phones, e-commerce, voting machines, ...
 - **Multimedia.** MP3, JPG, DivX, HDTV, face recognition, ...
 - **Social networks.** Recommendations, news feeds, advertisements, ...
 - **Physics.** N-body simulation, particle collision simulation, ...



立志成才 报国裕民



Typical Undergraduate Algorithm Course

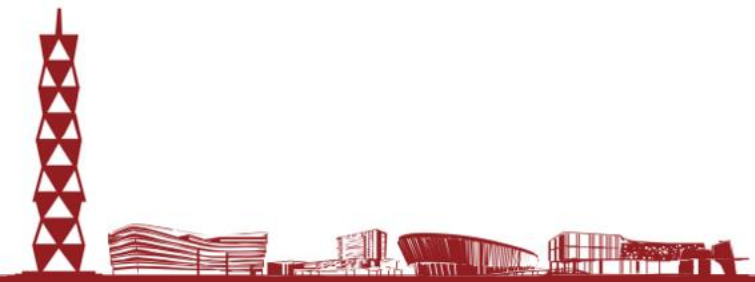


上海科技大学
ShanghaiTech University

Understanding and implementing classic algorithms

- Sorting
- Searching
- String algorithms
- Graph algorithms

Critical thinking, problem-solving, coding



立志成才 报效国家

Design and analysis of computer algorithms

- Vocabulary for design and analysis of algorithms
- Greedy algorithms
- Divide-and-conquer
- Dynamic programming
- Network flow
- Intractability (complexity classes)
- Amortized analysis
- Approximation algorithms
- Randomized algorithms
- Local search

Critical thinking, problem-solving, rigorous analysis





Integer Multiplication

- Input: two n -digit numbers x and y
- Output: the product $x \cdot y$
- “Primitive operation”: add or multiply two single-digit numbers
- The grade-school algorithm: $5678 \cdot 1234 = 7006652$
- **$2n$** operations per row and there are **n** rows
- Upshot: #operations overall $\leq \text{constant} \cdot n^2$

$$\begin{array}{r} \text{2} \text{ 2} \text{ 3} \text{ 3} \\ 5678 \\ \times 1234 \\ \hline 22712 \\ 17034- \\ 11356-- \\ 5678--- \\ \hline 7006652 \end{array}$$

- The Algorithm Designers' Mantra

“Perhaps the most important principle for the good algorithm designers is to refuse to be content.” – Aho, Hopcroft, and Ullman, The Design and Analysis of Computer Algorithms, 1974

- **CAN WE DO BETTER?**





Karatsuba Multiplication



- Example: $x = 5678$ $y = 1234$ to compute the product $x \cdot y$
- Assume $56 = a$, $78 = b$, $12 = c$, $34 = d$
- Step 1: Compute $a \cdot c = 672$
- Step 2: Compute $b \cdot d = 2652$
- Step 3: Compute $(a+b) \cdot (c+d) = 134 \cdot 46 = 6164$
- Step 4: Compute Step3 - step2 - step1 = 2840

$$5678 = 56 \cdot 100 + 78$$

$$1234 = 12 \cdot 100 + 34$$

$$x \cdot y = (ac) \cdot 10000 + (ad + bc) \cdot 100 + bd$$

$$\begin{array}{r} 6720000 \\ 2652 \\ 284000 \\ \hline 7006652 \end{array}$$





A Recursive Algorithm



- Write $x = 10^{n/2}a + b$ and $y = 10^{n/2}c + d$ where a, b, c, d are $n/2$ -digit numbers
- [Example: $a = 56, b = 78, c = 12, d = 34$]
- Then: $x \cdot y = (10^{n/2}a + b) \cdot (10^{n/2}c + d) = 10^n ac + 10^{n/2}(ad + bc) + bd$
- Idea; recursively compute ac, ad, bc, bd , then compute the above equation in the straightforward way
- Simple base case omitted (if input is very small, get the result immediately)

- **Karatsuba Multiplication**

$$(a + b)(c + d) = ac + ad + bc + bd \quad ad + bc = (a + b)(c + d) - ac - bd$$

- Recall: $x \cdot y = 10^n ac + 10^{n/2}(ad + bc) + bd$ (seems having 4 recursive multiplications...)

- Step 1: recursively compute ac

$$xy = ac \cdot 10000 + [(a + b)(c + d) - ac - bd] \cdot 100 + bd$$

- Step 2: recursively compute bd

- Step 3: recursively compute $(a+b)(c+d) = ac+ad+bc+bd$

- **Gauss's trick: step3 - step1 - step2 = ad + bc**

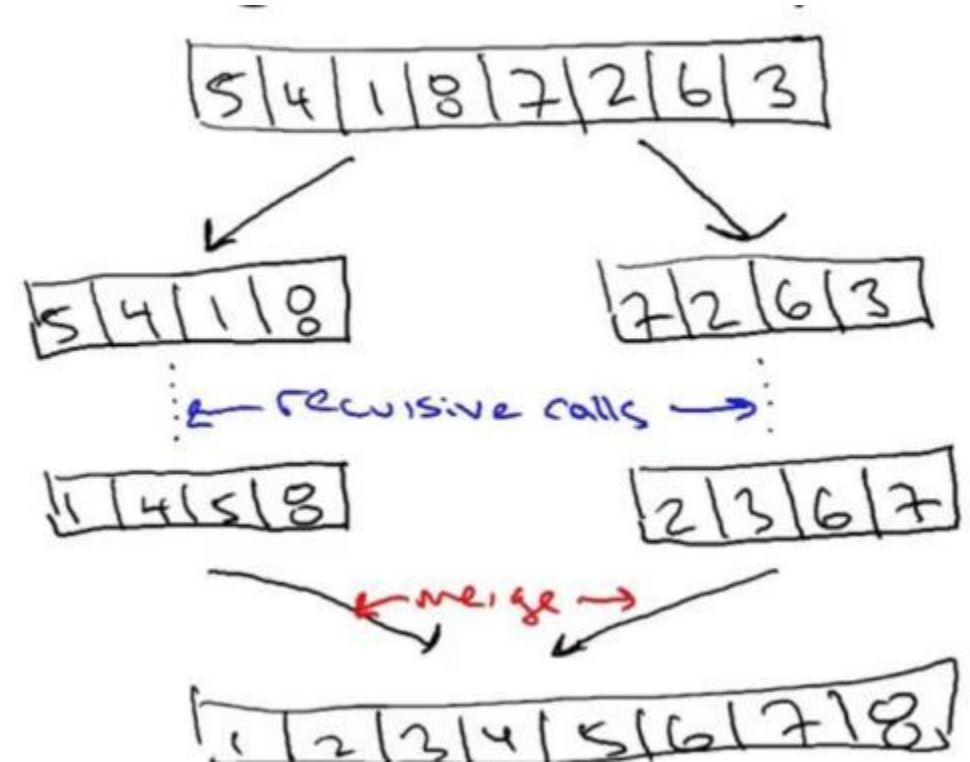
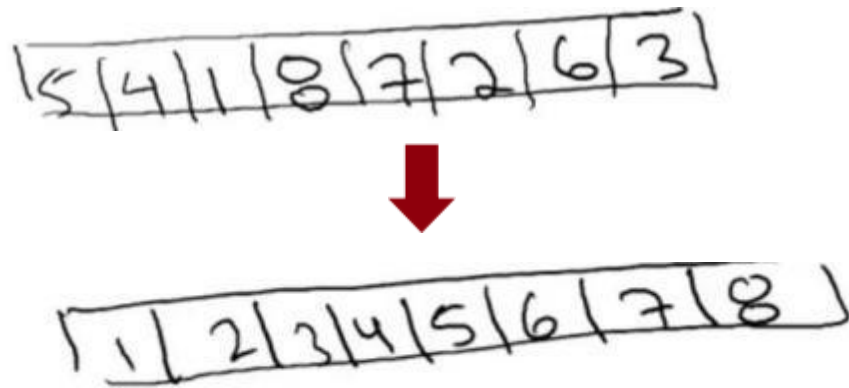
- Upshot: only need 3 recursive multiplications and some additions



Why study merge sort?

- Good introduction to divide & conquer
- Improves over selection, insertion, bubble sorts
- Motivates guiding principles for algorithm analysis (worst-case and asymptotic analysis)
- Analysis generalizes to “Master Method”

- The Sorting Problem
- Input: array of n numbers, unsorted
- Output: Same numbers, sorted in increasing order





Merge Sort: Pseudocode



- Recursively sort 1st half of input array
- Recursively sort 2nd half of input array
- Merge two sorted sublists into one

- Pseudocode for Merge:

C = output array [length = n]

A = 1st **sorted** array [n/2]

B = 2nd **sorted** array [n/2]

i = 1

j = 1

```
for k = 1 to n
```

```
  if A[i] < B[j]
```

```
    C[k] = A[i]
```

```
    i++
```

```
  else B[j] < A[i]
```

```
    C[k] = B[j]
```

```
    j++
```

```
End
```

(ignores end cases)



Merge Sort Running Time?

- **Key question:** running the MergeSort on array of n numbers?
- Running time $\approx$ # of lines of code executed

- Pseudocode for Merge:

C = output array [length = n]

A = 1st sorted array [$n/2$]

B = 2nd sorted array [$n/2$]

$i = 1, j = 1$

2 operations

```
for k = 1 to n
  if  $A[i] < B[j]$ 
     $C[k] = A[i]$ 
     $i++$ 
  else  $B[j] < A[i]$ 
     $C[k] = B[j]$ 
     $j++$ 
End
(ignore end cases)
```

- ① 比较一次
 $A[i] < B[j]$
→ 1 次操作
- ② 赋值一次
 $C[k] = \dots$
→ 1 次操作
- ③ 增加指针一次
 $i++$ 或 $j++$
→ 1 次操作
- ④ 循环控制
 $k++$ 和循环判断
→ 算 1 次

- **Upshot:** running time of merge on array of n numbers is $\leq 4n + 2 \leq 6n$ (since $n \geq 1$)
- **Claim:** MergeSort requires $\leq 6n \log n + 6n$ operations to sort n numbers. ($6n(\log n + 1)$)
- **Recall:** $\log n$ = # of times you divide by 2 until you get down to 1



Proof of claim (assuming $n = \text{power of } 2$)



- Will use "recursion tree"
- Q: Roughly how many levels does this recursion tree have (as a function of n , the length of the input array)?



- A constant number (independent of n)
- $\sqrt{n}$
- $\log_2 n$
- n



- Q: What is the pattern? Fill in blanks in the following statement: at each level $j = 0, 1, 2, \dots, \log_2 n$, there are ___ subproblems, each of size ____.



- 2^j and 2^j , respectively
- $n/2^j$ and $n/2^j$, respectively
- 2^j and $n/2^j$, respectively
- $n/2^j$ and 2^j , respectively





Proof of claim (assuming $n = \text{power of } 2$)



- At each level $j = 0, 1, 2, \dots, \log_2 n$, there are 2^j subproblems, each of size $n/2^j$
- Total # of operations at level j :
[each $j=0, 1, 2, \dots, \log_2 n$]
 $\leq 2^j * 6(n/2^j) = 6n \leftarrow \text{independent of } j$
- Total $\leq 6n(\log_2 n + 1)$
- **Claim:** For every input array of n numbers, MergeSort produces a sorted output array and uses at most $6n\log_2 n + 6n$ operations



Guiding Principles

1. Guiding Principle 1

- “worst-case analysis”: running time bound holds for every input of length n
- Particularly appropriate for “general-purpose” routines
- As opposed to “average-case” analysis and benchmarks (requires domain knowledge)
- Worst case usually easier to analyze

2. Guiding Principle 2

- Won't pay much attention to constant factors, lower-order terms
- Way easier
- Constants depend on architecture/compiler/programmer anyways
- Lose very little predictive power

3. Guiding Principle 3

- Asymptotic analysis: focus on running time for large input size n
- E.g., $6n\log_2 n + 6n$ “better than” $\frac{1}{2}n^2$ (e.g., insertion sort)
- Justification: any big problems are interesting

Fast algorithm:

worst-case running time grows slowly with input size



Five Representative Problems

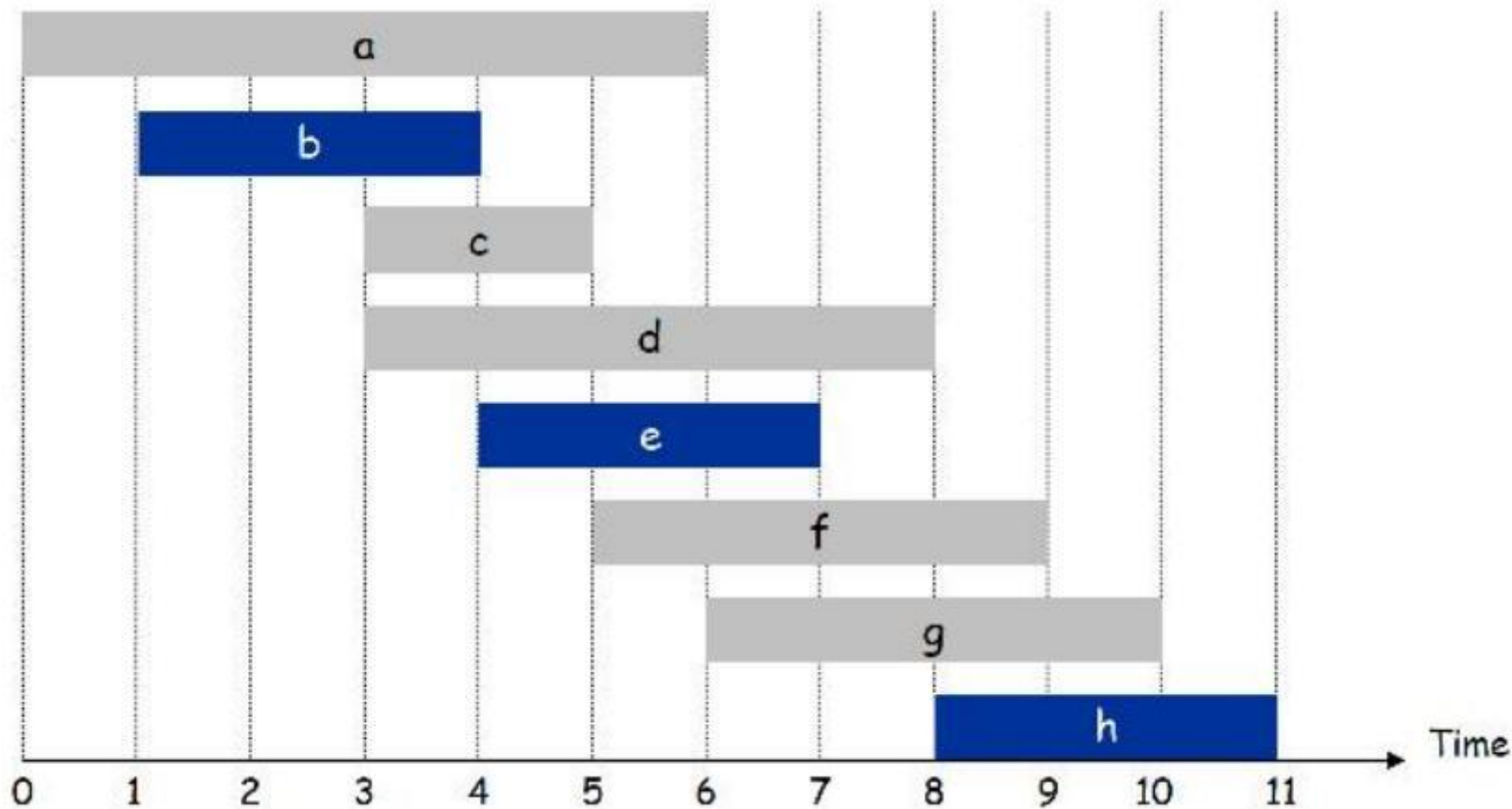




Interval Scheduling



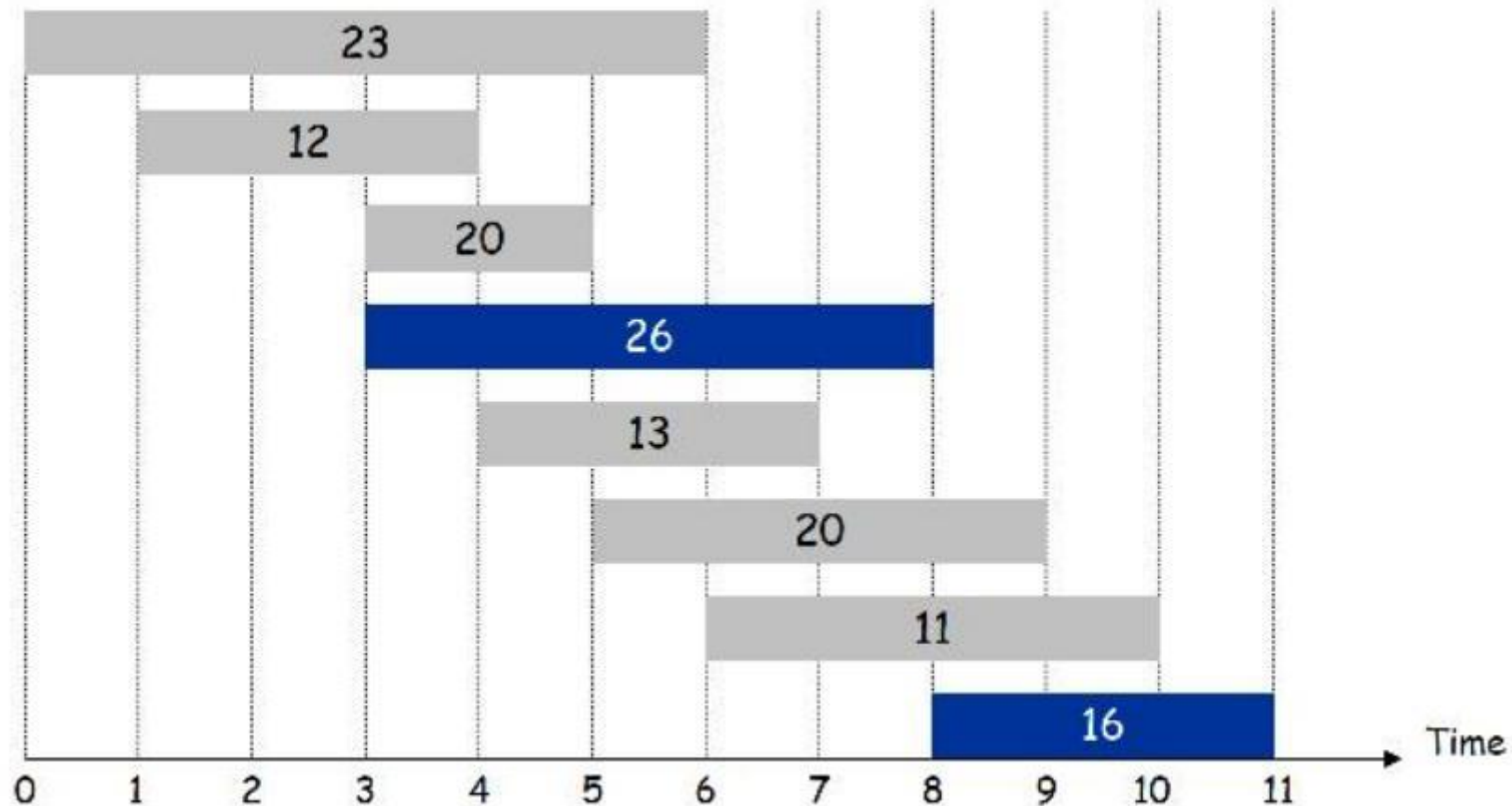
- **Input:** Set of jobs with start times and finish times
- **Goal:** Find **maximum cardinality** subset of mutually compatible (i.e., jobs don't overlap) jobs





Weighted Interval Scheduling

- **Input:** Set of jobs with start times, finish times, and weights
- **Goal:** Find maximum weight subset of mutually compatible jobs

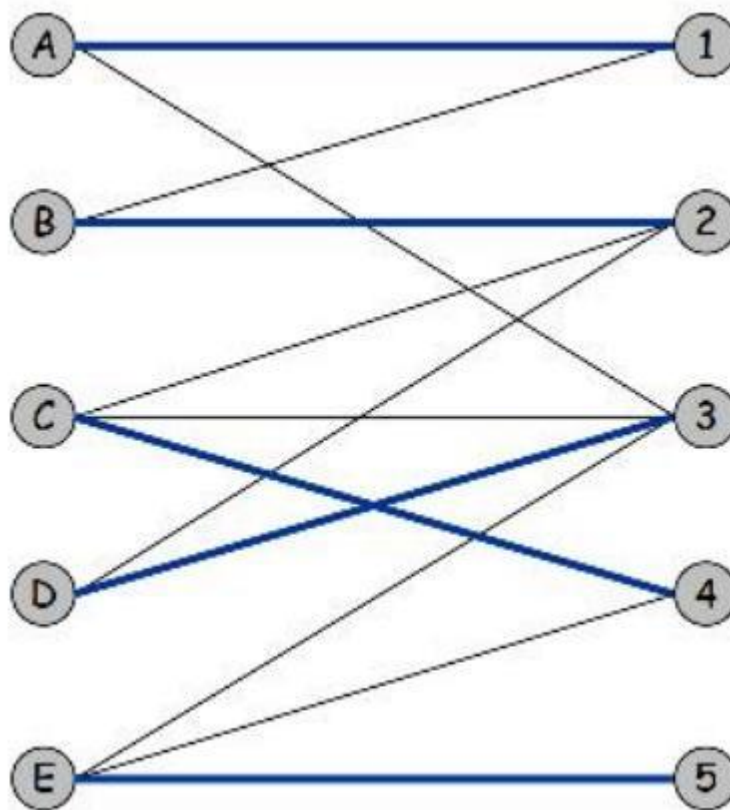




Bipartite Matching

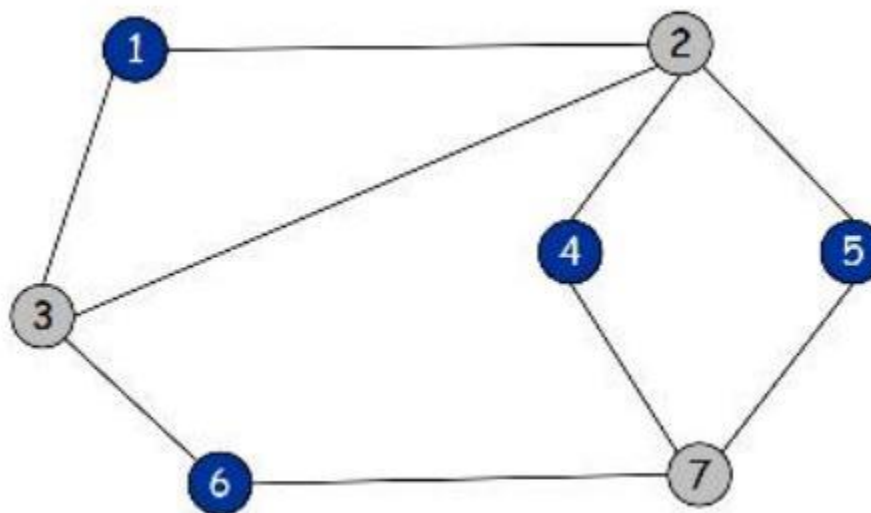


- **Input:** Bipartite graph
- **Goal:** Find **maximum cardinality** matching



Independent Set

- **Input:** Graph
- **Goal:** Find **maximum cardinality** independent (i.e., subset of nodes such that no two joined by an edge) set



- **Extension:** Weighted independent set

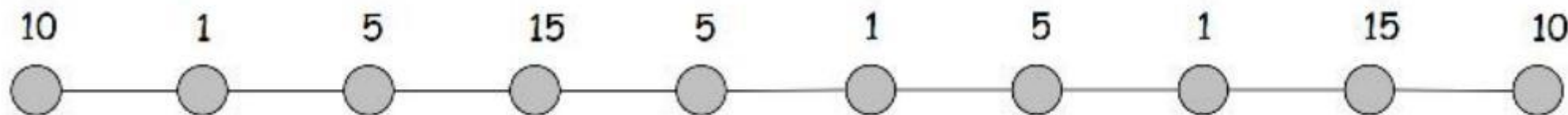




Competitive Facility Location



- **Input:** Graph with weight on each node
- **Game:** Two competing players alternate in selecting nodes. Not allowed to select a node if any of its neighbors have been selected
- **Goal:** Select a **maximum weight** subset of nodes



Second player can guarantee 20, but not 25





Five Representative Problems



- **Variations on a theme:** independent set
- **Interval scheduling:** $n \log n$ greedy algorithm
- **Weighted interval scheduling:** $n \log n$ dynamic programming algorithm
- **Bipartite matching:** n^2 max-flow-based algorithm
- **Independent set:** NP-complete
- **Competitive facility location:** PSPACE-complete





CS240 Algorithm Design and Analysis

Lecture 1

Computational Tractability

Qingying Hao
Spring 2026
2026.03.02



"For me, great algorithms are the poetry of computation. Just like verse, they can be terse, allusive, dense, and even mysterious. But once unlocked, they cast a brilliant new light on some aspect of computing." – Francis Sullivan, Science, Vol. 287, No. 5454, p.799, February 2000



Brute force. For many nontrivial problems, there is a natural brute-force search algorithm that checks every possible solution.

- Typically takes 2^n steps (or worse) for inputs of size n .
- Unacceptable in practice.



Ex. Stable matching problem: test all $n!$ perfect matchings for stability.

Desirable scaling property. When the input size doubles, the algorithm should slow down by at most some multiplicative constant factor C .

Def. An algorithm is **poly-time** if the above scaling property holds.

There exist constants $a > 0$ and $b > 0$ such that,
for every input of size n , the algorithm performs
 $\leq a n^b$ primitive computational steps.

corresponds
to $C = 2^b$



von Neumann
(1953)



Nash
(1955)



Gödel
(1956)



Cobham
(1964)



Edmonds
(1965)



Rabin
(1966)





Polynomial running time

We say that an algorithm is **efficient** if it has a polynomial running time.

Theory. Definition is (relatively) insensitive to model of computation.

Practice. It really works!

- The poly-time algorithms that people develop have both small constants and small exponents.
- Breaking through the exponential barrier of brute force typically exposes some crucial structure of the problem.

Exceptions. Some poly-time algorithms in the wild have galactic constants and/or huge exponents.

Q. Which would you prefer: $20 n^{120}$ or $n^{1 + 0.02 \ln n}$?

n^{120}



Map graphs in polynomial time

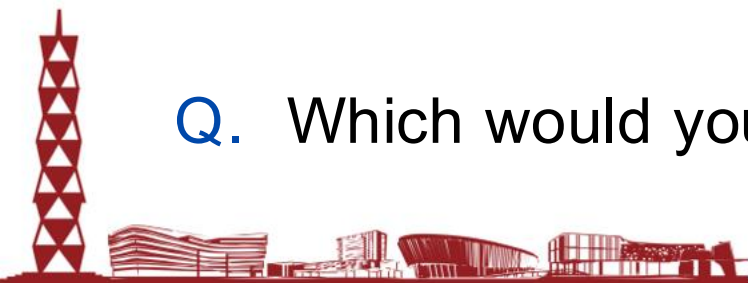
Mikkel Thorup*

Department of Computer Science, University of Copenhagen
Universitetsparken 1, DK-2100 Copenhagen East, Denmark
mthorup@diku.dk

Abstract

Chen, Grigni, and Papadimitriou (WADS'97 and STOC'98) have introduced a modified notion of planarity, where two faces are considered adjacent if they share at least one point. The corresponding abstract graphs are called map graphs. Chen et.al. raised the question of whether map graphs can be recognized in polynomial time. They showed that the decision problem is in NP and presented a polynomial time algorithm for the special case where we allow at most 4 faces to intersect in any point — if only 3 are allowed to intersect in a point, we get the usual planar graphs.

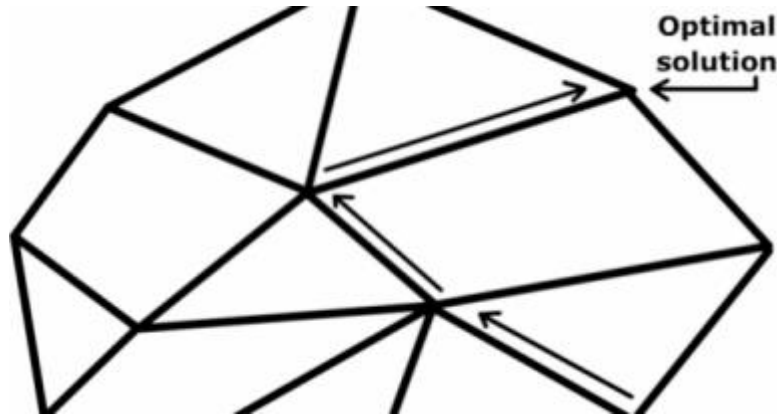
Chen et.al. conjectured that map graphs can be recognized in polynomial time, and in this paper, their conjecture is settled.



Worst case. Running time guarantee for **any input** of size n .

- Generally captures efficiency in practice.
- Draconian view, but hard to find effective alternative.

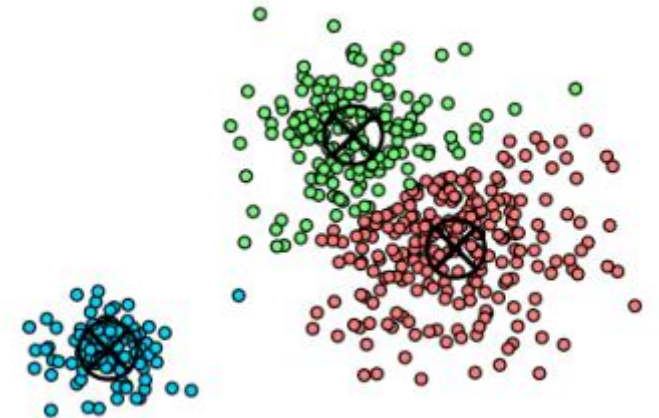
Exceptions. Some exponential-time algorithms are used widely in practice because the worst-case instances don't arise.



simplex algorithm



Linux grep



k-means algorithm



Other types of analyses

Probabilistic. Expected running time of a randomized algorithm.

Ex. The expected number of compares to quicksort n elements is $\sim 2n \ln n$.



Amortized. Worst-case running time for any sequence of n operations.

Ex. Starting from an empty stack, any sequence of n push and pop operations takes $O(n)$ primitive computational steps using a resizing array.

Also. Average-case analysis, smoothed analysis, competitive analysis, ...





Asymptotic Order of Growth

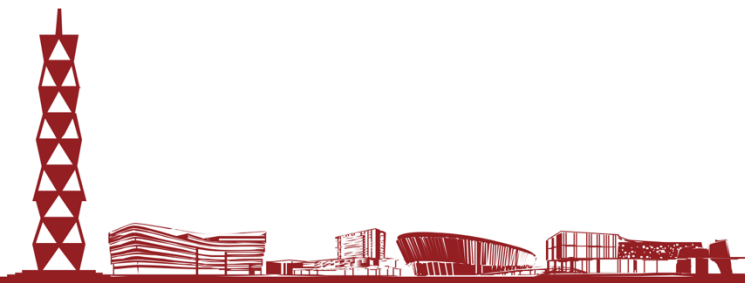




Asymptotic Order of Growth



- **Importance:** vocabulary for the design and analysis of algorithms (e.g., bit-oh notation)
- Sweet spot for high-level reasoning about algorithms
- Coarse enough to suppress architecture/language/compiler-dependent details
- Sharp enough to make useful comparisons between different algorithms, especially on large inputs (e.g., sorting or integer multiplication)
- High-level idea: Suppress constant factors (too system-dependent) and lower-order terms
→ irrelevant for large inputs
- **Example:** equate $6n\log_2 n + 6n$ with just $n\log n$
- **Terminology:** running time is $O(n\log n)$ where n = input size (e.g., length of input array)





Examples



- Problem: does array A contain the integer t?
- Given A (array of length n) and t (an integer)

```
for i = 1 to n
    if A[i] == t return TRUE
return FALSE
```

- Problem: Given A, B (array of length n) and t (an integer), does A or B contain t?

```
for i = 1 to n
    if A[i] == t return TRUE
for i = 1
    if B[i] == t return TRUE
return FALSE
```

- Problem: Do arrays A and B have a number in common?

```
for i = 1 to n
    for j = 1 to n
        if A[i] == B[j] return TRUE
return FALSE
```

- Problem: Do arrays A have duplicated entities?

```
for i = 1 to n
    for j = i+1 to n
        if A[i] == A[j] return TRUE
return FALSE
```

Question: What is the running time?

O(1)
O(n)
O(logn)
O(n²)





Asymptotic Order of Growth



- **Upper bounds.** $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \leq c \cdot f(n)$
- Example #1
 - Claim: If $T(n) = a_k n^k + \dots + a_1 n + a_0$ then $T(n) = O(n^k)$
 - Proof: Choose $n_0 = 1$ and $c = |a_k| + |a_{k-1}| + \dots + |a_1| + |a_0|$
 - Need to show that exist $n \geq 1$, $T(n) \leq cn^k$
 - We have for every $n \geq 1$, $T(n) \leq |a_k|n^k + \dots + |a_1|n + |a_0| \leq |a_k|n^k + \dots + |a_1|n^k + |a_0|n^k = cn^k$
- Example #2
 - Claim: for every $k \geq 1$, n^k is not $O(n^{k-1})$
 - Proof: by contradiction. Suppose $n^k = O(n^{k-1})$
 - Then exist constants $c, n_0 > 0$ such that $n^k \leq cn^{k-1}$ for every $n \geq n_0$
 - But then cancelling n^{k-1} from both sides $\rightarrow n \leq c$ which is clearly false





Asymptotic Order of Growth



- **Upper bounds.** $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \leq c \cdot f(n)$
- **Lower bounds.** $T(n)$ is $\Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \geq c \cdot f(n)$
- **Tight bounds.** $T(n)$ is $\Theta(f(n))$ if $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$.
 - Exist constants c_1, c_2, n_0 , such that $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n \geq n_0$
- Let $T(n) = \frac{1}{2}n^2 + 3n$. Which of the following statements are true?
 - $T(n) = O(n)$
 - $T(n) = \Omega(n)$
 - $T(n) = \Theta(n)$
 - $T(n) = O(n^3)$

Ex: $T(n) = 32n^2 + 17n + 32$
 $T(n)$ is $O(n^2), O(n^3) \leftarrow$ choose $c = 50, n_0 = 1$
 $T(n)$ is $\Omega(n^2), \Omega(n) \leftarrow$ choose $c=32, n_0 = 1$
 $T(n)$ is $\Theta(n^2)$
 $T(n)$ is not $O(n), \Omega(n^3), \Theta(n)$, or $\Theta(n^3)$



More Examples



- **Claim:** $2^{n+10} = O(2^n)$
- **Proof:** need to pick constants c, n_0 such that $2^{n+10} \leq c2^n$ for every $n \geq n_0$
- **Note:** $2^{n+10} = 2^{10}2^n = (1024)2^n$
- **So:** if we choose $c = 1024, n_0 = 1$, then $2^{n+10} \leq c2^n$ holds

- **Claim:** 2^{10n} is not $O(2^n)$
- **Proof:** By contradiction. If $2^{10n} = O(2^n)$, then exist constants $c, n_0 > 0$, such that
 - $2^{10n} \leq c2^n$ for every $n \geq n_0$
 - But then cancelling 2^n :
 - $2^{9n} \leq c$ for every $n \geq n_0$ which is certainly false



More Examples



- **Claim:** for every pair of (positive) functions $f(n)$, $g(n)$, $\max\{f, g\} = \Theta(f(n) + g(n))$
- **Proof:** $\max\{f, g\} = \Theta(f(n) + g(n))$

For every n , we have $\max\{f(n), g(n)\} \leq f(n) + g(n)$

and

$$2\max\{f(n), g(n)\} \geq f(n) + g(n) \rightarrow \max\{f(n), g(n)\} \geq \frac{1}{2} (f(n) + g(n))$$

Thus: $\frac{1}{2} (f(n) + g(n)) \leq \max\{f(n), g(n)\} \leq f(n) + g(n)$ for all $n \geq 1$

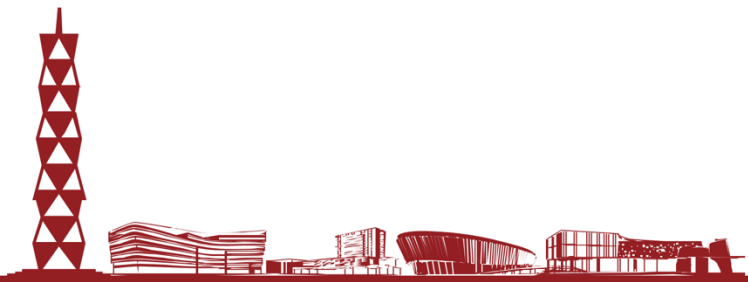
$\rightarrow \max\{f, g\} = \Theta(f(n) + g(n))$, where $n_0 = 1$, $c_1 = \frac{1}{2}$, $c_2 = 1$

$$f(n) + g(n) \leq 2 \max\{f(n), g(n)\}$$

$$\max\{f(n), g(n)\} \geq \frac{1}{2} (f(n) + g(n))$$

$$\max\{f, g\} = \Omega(f + g)$$

$$\frac{1}{2} (f + g) \leq \max\{f, g\} \leq (f + g)$$





Notation and Properties



- **Slight abuse of notation.** $T(n) = O(f(n))$

- **Asymmetric**

- $f(n) = 5n^3$; $g(n) = 3n^2$
- $f(n) = O(n^3) = q(n)$
- $g(n) = O(f(n))$ $f(n) \neq O(g(n))$
- Better notation: $T(n) \in O(f(n))$

$O(f(n))$ is a set of functions, but we often write $T(n) = O(f(n))$ instead of $T(n) \in O(f(n))$

- **Transitivity**

- If $f=O(g)$ and $g=O(h)$ then $f=O(h)$
- If $f= \Omega(g)$ and $g= \Omega(h)$ then $f= \Omega(h)$
- If $f= \Theta(g)$ and $g= \Theta(h)$ then $f= \Theta(h)$

- **Additivity**

- If $f = O(h)$ and $g = O(h)$ then $f + g = O(h)$
- If $f = \Omega(h)$ and $g = \Omega(h)$ then $f + g = \Omega(h)$
- If $f = \Theta(h)$ and $g = \Theta(h)$ then $f + g = \Theta(h)$





Asymptotic Bounds for Some Common Functions

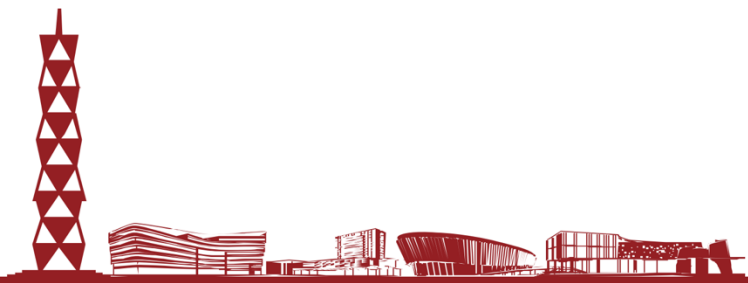


- **Polynomials.** $a_0 + a_1n + \dots + a_dn^d$ is $\Theta(n^d)$ if $a_d > 0$
- **Polynomial time.** Running time is $O(n^d)$ for some constant d independent of the input size n
- **Logarithms.** $O(\log_a n) = O(\log_b n)$ for any constants $a, b > 1$
- **Logarithms.** For every $x > 0$, $\log n = O(n^x)$

log grows slower than every polynomial

- **Exponentials.** For every $r > 1$ and every $d > 0$, $n^d = O(r^n)$

every exponential grows faster than every polynomial





A Survey of Common Running Times



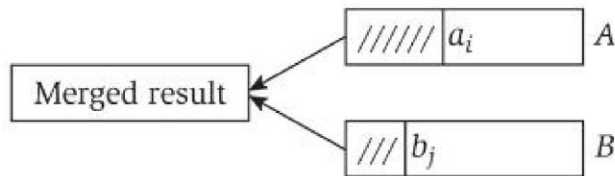
Linear Time: $O(n)$



- **Linear time.** Running time is at most a constant factor times the size of the input
- **Computing the maximum.** Compute the maximum of n numbers $a_1, \dots, a_n$

```
max ← a1
for i = 2 to n {
    if (ai > max)
        max ← ai
}
```

- **Merge.** Combine two sorted lists $A = a_1, a_2, \dots, a_n$ with $B = b_1, b_2, \dots, b_n$ into sorted whole



```
i = 1, j = 1
while (both lists are nonempty) {
    if (ai ≤ bj) append ai to output list and increment i
    else         append bj to output list and increment j
}
append remainder of nonempty list to output list
```

- **Claim.** Merging two lists of size n takes $O(n)$ time
- **Pf.** After each comparison, the length of output list increases by 1

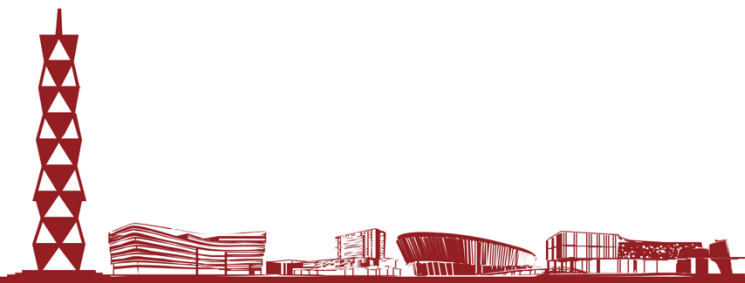




$O(n \log n)$ Time



- **$O(n \log n)$ time.** Arises in divide-and-conquer algorithms
- **Sorting.** Mergesort and heapsort are sorting algorithms that perform $O(n \log n)$ comparisons
- **Largest empty interval.** Given n timestamps $x_1, \dots, x_n$ on which copies of a file arrive at a server, what is largest interval of time when no copies of the file arrive?
- **$O(n \log n)$ solution.** Sort the timestamps. Scan the sorted list in order, identifying the maximum gap between successive timestamps





Quadratic Time: $O(n^2)$



- **Quadratic time.** Enumerate all pairs of elements
- **Closest pair of points.** Given a list of n points in the plane $(x_1, y_1), \dots, (x_n, y_n)$, find the pair that is closest
- **$O(n^2)$ solution.** Try all pairs of points

```
min  $\leftarrow (x_1 - x_2)^2 + (y_1 - y_2)^2$ 
for i = 1 to n {
    for j = i+1 to n {
        d  $\leftarrow (x_i - x_j)^2 + (y_i - y_j)^2$ 
        if (d < min)
            min  $\leftarrow$  d
    }
}
```

Don't need to take square roots

- **Remark.** $\Omega(n^2)$ seems inevitable, but this is just an illusion





Efficient implementation

Goal. Implement Gale–Shapley to run in $O(n^2)$ time.

GALE–SHAPLEY (*preference lists for n hospitals and n students*)

Initialize all hospitals and students as unmatched.

While there exists an unmatched hospital h :

 Let s be the first student on h 's list
 to whom h has not yet proposed.

 If s is unmatched

 Match h with s

 Else if s prefers h to her current partner h'

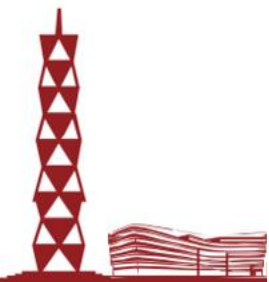
 Unmatch h' and s

 Match h with s

 Else

s rejects h

Return all matched pairs



Cubic Time: $O(n^3)$



- **Cubic time.** Enumerate all triples of elements
- **Set disjointness.** Given n sets $S_1, \dots, S_n$ each of which is a subset of $1, 2, \dots, n$, is there some pair of these which are disjoint?
- **$O(n^3)$ solution.** For each pairs of sets, determine if they are disjoint

```
foreach set  $S_i$  {  
    foreach other set  $S_j$  {  
        foreach element  $p$  of  $S_i$  {  
            determine whether  $p$  also belongs to  $S_j$   
        }  
        if (no element of  $S_i$  belongs to  $S_j$ )  
            report that  $S_i$  and  $S_j$  are disjoint  
    }  
}
```





Polynomial Time: $O(n^k)$ Time



- **Independent set of size k.** Given a graph, are there k nodes such that no two are joined by an edge?
- **$O(n^k)$ solution.** Enumerate all subsets of k nodes

```
foreach subset S of k nodes {  
    check whether S is an independent set  
    if (S is an independent set)  
        report S is an independent set  
    }  
}
```

- Check whether S is an independent set = $O(k^2)$
- Number of k element subsets =
$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots(2)(1)} \leq \frac{n^k}{k!}$$
- $O(k^2 n^k / k!) = O(n^k)$

poly-time for $k=17$,
but not practical



Exponential Time



- **Independent set.** Given a graph, what is maximum size of an independent set?
- **$O(n^2 2^n)$ solution.** Enumerate all subsets.

```
S* ←  $\phi$ 
foreach subset S of nodes {
    check whether S is an independent set
    if (S is largest independent set seen so far)
        update S* ← S
}
```

